

Document number: P1450R1
Revises: P1450R0
Date: 2019-06-17
Project: ISO JTC1/SC22/WG21: Programming Language C++
Audience: LEWG
Reply to: Vincent Reverdy
University of Illinois at Urbana-Champaign
vince.rev@gmail.com

Enriching type modification traits

Note: this is an early draft. It's known to be incomplet and incorrekt, and it has lots of bad fomattting.

Abstract

We introduce additional type traits to the standard library focused on type modification. The new type traits we present considerably simplify qualifiers manipulation. We also introduce a new type trait to remove all pointers on a type for the sake of completeness. These type traits have been especially useful in the design of proxy classes, included an updated design for bit manipulation utilities. They also have been used extensively in the implementation of a library dedicated to the creation of custom overload sets that will be proposed for standardization in a separate proposal.

Contents

1	Proposal	1
1.1	History	1
1.2	Introduction	1
1.3	Impact on the standard	1
1.4	Motivations and design decisions	1
1.4.1	Pointers removal	1
1.4.2	Qualifiers manipulation	2
1.4.3	Combined removal and qualifiers manipulation	3
1.5	Technical specification	4
1.6	Discussion and open questions	4
1.6.1	Bikeshedding	4
1.7	Acknowledgements	4
1.8	References	5
2	Wording	6
2.1	Metaprogramming and type traits	6
2.1.1	Requirements	6
2.1.2	Header <code><type_traits></code> synopsis	6
2.1.3	Helper classes	8
2.1.4	Unary type traits	8
2.1.5	Type property queries	8
2.1.6	Relationships between types	8
2.1.7	Transformations between types	8
2.1.8	Logical operator traits	10
2.1.9	Endian	10

1 Proposal

[proposal]

1.1 History

[proposal.history]

- **P1450R0** was carefully reviewed by LEWGI at the Kona 2019 Standards Committee Meeting, and was approved to be forwarded to LEWG with minor modifications. Jonathan Wakely and Eric Fiselier both confirmed that they have their own implementation of some of these type traits as an internal detail of their codebase. LEWGI recommended the removal of the `clone_*` form, as well as `copy_signedness`, and suggested the removal of `copy_all_extents` and `copy_all_pointers`. For consistency with the rest of the type traits, the last two are kept for now, and will be removed if LEWG wants to. LEWGI did not find a better name than `copy_*` to name these type traits.
- At the San Diego 2018 Standards Committee Meeting, LEWGI recommended the extraction of simple type modification traits from the original **P1016R0** proposal, which led to the proposal **P1450R0** in its current form.
- A first proposal was submitted in 2018 as **P1016R0**. However, back then it was not clear whether this kind of type traits should wait for reflection. In San Diego 2018, SG7 clarified that these type traits are pure library facilities that do not need to be first reviewed by them. They also clarified the fact that type traits in their current form live in a different space than reflection, and that the second one will not make the first one disappear. As a consequence, basic type traits should not wait for reflection. In San Diego, LEWGI recommended the extraction of simple type modification traits from the original **P1016R0** proposal, which led to this proposal in its current form.
- Originally developed as helper traits for an overload sets and sequences library, presented at CppCon 2018 (**Custom Overload Sets and Inline SFINAE for Truly Generic Interfaces**). Early discussions and feedback included a thread entitled **General purpose utilities for template metaprogramming and type manipulation** on the future proposals Google group.

1.2 Introduction

[proposal.introduction]

Since their introduction with C++11, the standard library type traits have been of great help for template metaprogramming. They contributed to the standardization of common metaprogramming patterns, such as SFINAE with `enable_if`, and since C++17 with `void_t`. In this paper, we introduce new type traits corresponding to metaprogramming patterns that turned out to be very useful to implement template proxy classes as well as to implement a tool to build custom overload sets. This tool will be proposed for standardization in a separate paper. We believe that the listed type traits are of common use and could benefit the entire community. The new type traits fall in three different categories:

- pointers removal: `remove_all_pointers` inspired from `remove_all_extents`
- qualifiers manipulation: `copy_*` type traits
- ~~combined removal and qualifiers manipulation: `clone_*` type traits~~

An implementation is available at <https://github.com/vreverd/type-utilities>.

1.3 Impact on the standard

[proposal.impact]

This proposal is a pure library extension. It does not require changes to any standard classes or functions. All the extensions belong to the `<type_traits>` header.

1.4 Motivations and design decisions

[proposal.design]

1.4.1 Pointers removal

[proposal.design.remove]

```
// Pointers removal
template <class T> struct remove_all_pointers;

// Type alias
template <class T> using remove_all_pointers_t = typename remove_all_pointers<T>::type;
```

The current standard library includes two type traits to manipulate extents: `remove_extent` which removes the first array dimension, and `remove_all_extents` which removes all dimensions. For pointers, only one is currently provided: `remove_pointer` which removes one pointer. However, in some contexts it can be useful to access the “raw” type

However for the same reason that it can be useful to remove all dimensions, it can sometimes be useful to remove all pointers and access the “raw” type. Also, in the context of qualifiers manipulation (see (1.4.2) and (1.4.3)), it makes sense to provide tools to transform a `int***` into a `double***` by transferring all pointers from one type to another: `copy_all_pointers` ~~and `clone_all_pointers`~~. In this context, being able to remove all pointers seems to be a natural addition to the standard library, for completeness. For all these reasons, we propose to introduce the type trait: `remove_all_pointers`.

1.4.2 Qualifiers manipulation

[proposal.design.copy]

```
// Qualifiers manipulation
template <class From, class To> struct copy_const;
template <class From, class To> struct copy_volatile;
template <class From, class To> struct copy_cv;
template <class From, class To> struct copy_reference;
// template <class From, class To> struct copy_signedness;
template <class From, class To> struct copy_extent;
template <class From, class To> struct copy_all_extents;
template <class From, class To> struct copy_pointer;
template <class From, class To> struct copy_all_pointers;
template <class From, class To> struct copy_cvref;

// Type aliases
template <class F, class T> using copy_const_t = typename copy_const<F, T>::type;
template <class F, class T> using copy_volatile_t = typename copy_volatile<F, T>::type;
template <class F, class T> using copy_cv_t = typename copy_cv<F, T>::type;
template <class F, class T> using copy_reference_t = typename copy_reference<F, T>::type;
// template <class F, class T> using copy_signedness_t = typename copy_signedness<F, T>::type;
template <class F, class T> using copy_extent_t = typename copy_extent<F, T>::type;
template <class F, class T> using copy_all_extents_t = typename copy_all_extents<F, T>::type;
template <class F, class T> using copy_pointer_t = typename copy_pointer<F, T>::type;
template <class F, class T> using copy_all_pointers_t = typename copy_all_pointers<F, T>::type;
template <class F, class T> using copy_cvref_t = typename copy_cvref<F, T>::type;
```

In the heavy template metaprogramming involved in the building of template proxy classes and custom overload sets, one pattern happened to be very useful: being able to transfer the qualifiers of one type to another one. For example, to transform a `const int&` into a `const double&`, a `int[1][2][3]` into a `double[1][2][3]`, or an `int***` to a `double***`. It can be also used in a function taking a universal reference as an input, to qualify another type based on the qualification of the input:

```
template <class T> void f(T&& x) {
    // An integer with the same qualification as the input
    using integer = std::copy_cvref_t<T&&, int>;
    /* function contents */
}
```

or to make a type `const` depending on another type:

```
template <class T> struct foo {
    // Data members
    T a;
    std::copy_const_t<T, int> n;
    std::copy_const_t<T, double> x;
    /* class contents */
};
```

Another uses are illustrated in [P0847R0](#), where `copy_cvref_t` is called `like_t`.

For completeness, qualifier manipulators are added to all existing categories of type transformations: cv ([2.1.7.1](#)), reference ([2.1.7.2](#)), ~~sign ([2.1.7.3](#))~~, array ([2.1.7.4](#)) and pointer ([2.1.7.5](#)). Additionally, depending on the behavior regarding the second template parameter, two kinds of qualifier parameters are introduced: the copiers `copy_*` ~~and the cloners `clone_*`~~ presented in the next section.

The complete list of proposed `copy_*` traits is:

- const-volatile modifications: `copy_const`, `copy_volatile`, `copy_cv`
- reference modifications: `copy_reference`
- ~~sign modifications: `copy_signedness`~~
- array modifications: `copy_extent`, `copy_all_extents`
- pointer modifications: `copy_pointer` `copy_all_pointers`
- other transformations: `copy_cvref`

As a note, in the same way `remove_pointer` deals with cv-qualified pointers, `copy_pointer` `copy_all_pointers` copy the cv-qualifiers of pointers. ~~Also `copy_signedness` is preferred over `copy_sign` to avoid confusion with the existing mathematical function `copysign`.~~

1.4.3 Combined removal and qualifiers manipulation [proposal.design.clone]

```
// Combined removal and qualifiers manipulation
// template <class From, class To> struct clone_const;
// template <class From, class To> struct clone_volatile;
// template <class From, class To> struct clone_cv;
// template <class From, class To> struct clone_reference;
// template <class From, class To> struct clone_extent;
// template <class From, class To> struct clone_all_extents;
// template <class From, class To> struct clone_pointer;
// template <class From, class To> struct clone_all_pointers;
// template <class From, class To> struct clone_cvref;

// Type aliases
// template <class F, class T> using clone_const_t = typename clone_const<F, T>::type;
// template <class F, class T> using clone_volatile_t = typename clone_volatile<F, T>::type;
// template <class F, class T> using clone_cv_t = typename clone_cv<F, T>::type;
// template <class F, class T> using clone_reference_t = typename clone_reference<F, T>::type;
// template <class F, class T> using clone_signedness_t = typename clone_signedness<F, T>::type;
// template <class F, class T> using clone_extent_t = typename clone_extent<F, T>::type;
// template <class F, class T> using clone_all_extents_t = typename clone_all_extents<F, T>::type;
// template <class F, class T> using clone_pointer_t = typename clone_pointer<F, T>::type;
// template <class F, class T> using clone_all_pointers_t = typename clone_all_pointers<F, T>::type;
// template <class F, class T> using clone_cvref_t = typename clone_cvref<F, T>::type;
```

When the second template parameter is also coming from a context where it can be qualified, it can be useful to first remove its qualifiers before copying the new one. The difference between cloners and copiers is that the copiers directly copy the qualifiers of the first argument to the second, while cloners first discard the qual-

ifiers of the second argument. For example `copy_cv_t<volatile int, const double>` evaluates to `const volatile double` while `clone_cv_t<volatile int, const double>` evaluates to `volatile double`, and `copy_all_pointers_t<int***, double*>` evaluates to `double****` while `clone_all_pointers_t<int***, double*>` evaluates to `double***`.

For example:

```
template <class T> struct foo {
    // Function member
    template <class U> void bar (U&& x) {
        std::clone_cvref_t<T, U> something;
        /* function contents */
    }
    /* class contents */
};
```

For completeness, qualifier manipulators are added to all existing categories of type transformations: cv (2.1.7.1), reference (2.1.7.2), sign (2.1.7.3), array (2.1.7.4) and pointer (2.1.7.5).

The complete list of proposed `clone_*` traits is:

- ~~const-volatile modifications: `clone_const`, `clone_volatile`, `clone_cv`~~
- ~~reference modifications: `clone_reference`~~
- ~~array modifications: `clone_extent`, `clone_all_extents`~~
- ~~pointer modifications: `clone_pointer` `clone_all_pointers`~~
- ~~other transformations: `clone_cvref`~~

As a note, in the same way `remove_pointer` deals with cv-qualified pointers, `clone_pointer` `clone_all_pointers` clone the cv-qualifiers of pointers. Finally, `clone_signedness` is not introduced, because `remove_sign` does not exist, and does not seem to be a relevant type trait to introduce, the only interesting use case being to transform a `signed char` or an `unsigned char` into a `char`. The difference between `copy_signedness` and a hypothetical `clone_signedness` would be the following: `copy_signedness_t<char, unsigned char>` would evaluate to `unsigned char` while `clone_signedness_t<char, unsigned char>` would evaluate to `char`. In both cases `copy/clone_signedness_t<unsigned int, int>` would evaluate to `unsigned int` and `copy/clone_signedness_t<signed int, unsigned int>` would evaluate to `signed int`.

1.5 Technical specification

[proposal.spec]

See the wording (part 2).

1.6 Discussion and open questions

[proposal.discussion]

1.6.1 Bikeshedding

[proposal.discussion.bikeshed]

While some names are straightforward and follow existing patterns in standard library, the following names are the most likely to be debated:

- `copy_*`
- ~~`clone_*`~~
- ~~`copy_signedness`~~

1.7 Acknowledgements

[proposal.ackwldgmnts]

The authors would like to thank the participants to the related discussion on the [future-proposals](#) group. This work has been made possible thanks to the National Science Foundation through the awards CCF-1647432 and SI2-SSE-1642411.

1.8 References

[`proposal.references`]

[A few additional type manipulation utilities](#), Vincent Reverdy, *Github* (March 2018)

[P1016R0](#), A few additional type manipulation utilities, Vincent Reverdy, *ISO/IEC JTC1/SC22/WG21* (May 2018)

[N4727](#), Working Draft, Standard for Programming Language C++, Richard Smith, *ISO/IEC JTC1/SC22/WG21* (February 2018)

[P0847R0](#), Deducing this, Gasper Azman et al., *ISO/IEC JTC1/SC22/WG21* (February 2018)

[General purpose utilities for template metaprogramming and type manipulation](#), ISO C++ Standard - Future Proposals, *Google Groups* (March 2018)

2 Wording

[wording]

2.1 Metaprogramming and type traits

[meta]

2.1.1 Requirements

[meta.rqmts]

¹ No modification.

2.1.2 Header `<type_traits>` synopsis

[meta.type.synop]

¹ Add the following to the synopsis of `<type_traits>`:

```
namespace std {
    // 2.1.3, helper classes

    // 2.1.4.1, primary type categories

    // 2.1.4.2, composite type categories

    // 2.1.4.3, type properties

    // 2.1.5, type property queries

    // 2.1.6, type relations

    // 2.1.7.1, const-volatile modifications
    template <class From, class To> struct copy_const;
    // template <class From, class To> struct clone_const;
    template <class From, class To> struct copy_volatile;
    // template <class From, class To> struct clone_volatile;
    template <class From, class To> struct copy_cv;
    // template <class From, class To> struct clone_cv;

    template <class From, class To>
    using copy_const_t = typename copy_const<From, To>::type;
    // template <class From, class To>
    // using clone_const_t = typename clone_const<From, To>::type;
    template <class From, class To>
    using copy_volatile_t = typename copy_volatile<From, To>::type;
    // template <class From, class To>
    // using clone_volatile_t = typename clone_volatile<From, To>::type;
    template <class From, class To>
    using copy_cv_t = typename copy_cv<From, To>::type;
    // template <class From, class To>
    // using clone_cv_t = typename clone_cv<From, To>::type;

    // 2.1.7.2, reference modifications
    template <class From, class To> struct copy_reference;
    // template <class From, class To> struct clone_reference;

    template <class From, class To>
    using copy_reference_t = typename copy_reference<From, To>::type;
    // template <class From, class To>
```



```

// using clone_reference_t = typename clone_reference<From, To>::type;

// 2.1.7.3, sign modifications
// template <class From, class To> struct copy_signedness;

// template <class From, class To>
// using copy_signedness_t = typename copy_signedness<From, To>::type;

// 2.1.7.4, array modifications
template <class From, class To> struct copy_extent;
// template <class From, class To> struct clone_extent;
template <class From, class To> struct copy_all_extents;
// template <class From, class To> struct clone_all_extents;

template <class From, class To>
using copy_extent_t = typename copy_extent<From, To>::type;
// template <class From, class To>
// using clone_extent_t = typename clone_extent<From, To>::type;
template <class From, class To>
using copy_all_extents_t = typename copy_all_extents<From, To>::type;
// template <class From, class To>
// using clone_all_extents_t = typename clone_all_extents<From, To>::type;

// 2.1.7.5, pointer modifications
template <class T> struct remove_all_pointers;
template <class From, class To> struct copy_pointer;
// template <class From, class To> struct clone_pointer;
template <class From, class To> struct copy_all_pointers;
// template <class From, class To> struct clone_all_pointers;

template <class T>
using remove_all_pointers_t = typename remove_all_pointers<T>::type;
template <class From, class To>
using copy_pointer_t = typename copy_pointer<From, To>::type;
// template <class From, class To>
// using clone_pointer_t = typename clone_pointer<From, To>::type;
template <class From, class To>
using copy_all_pointers_t = typename copy_all_pointers<From, To>::type;
// template <class From, class To>
// using clone_all_pointers_t = typename clone_all_pointers<From, To>::type;

// 2.1.7.6, other transformations
template <class From, class To> struct copy_cvref;
// template <class From, class To> struct clone_cvref;

template <class From, class To>
using copy_cvref_t = typename copy_cvref<From, To>::type;
// template <class From, class To>
// using clone_cvref_t = typename clone_cvref<From, To>::type;

// 2.1.8, logical operator traits

// 2.1.9, endian
}

```

2.1.3 Helper classes [meta.help]

¹ No modification.

2.1.4 Unary type traits [meta.unary]

¹ No modification.

2.1.4.1 Primary type categories [meta.unary.cat]

¹ No modification.

2.1.4.2 Composite type traits [meta.unary.comp]

¹ No modification.

2.1.4.3 Type properties [meta.unary.prop]

¹ No modification.

2.1.5 Type property queries [meta.unary.prop.query]

¹ No modification.

2.1.6 Relationships between types [meta.rel]

¹ No modification.

2.1.7 Transformations between types [meta.trans]

2.1.7.1 Const-volatile modifications [meta.trans.cv]

¹ Add the following to the table “Const-volatile modifications”:

Table 1 — Const-volatile modifications

Template	Comments
<code>template<class From, class To> struct copy_const;</code>	The member typedef <code>type</code> names the same type as <code>add_const_t<To></code> if <code>is_const_v<From></code> , and <code>To</code> otherwise.
<code>template<class From, class To> struct clone_const;</code>	The member typedef <code>type</code> names the same type as <code>copy_const_t<From, remove_const_t<To>></code> .
<code>template<class From, class To> struct copy_volatile;</code>	The member typedef <code>type</code> names the same type as <code>add_volatile_t<To></code> if <code>is_volatile_v<From></code> , and <code>To</code> otherwise.
<code>template<class From, class To> struct clone_volatile;</code>	The member typedef <code>type</code> names the same type as <code>copy_volatile_t<From, remove_volatile_t<To>></code> .
<code>template<class From, class To> struct copy_cv;</code>	The member typedef <code>type</code> names the same type as <code>copy_const_t<From, copy_volatile_t<From, To>></code> .
<code>template<class From, class To> struct clone_cv;</code>	The member typedef <code>type</code> names the same type as <code>copy_cv_t<From, remove_cv_t<To>></code> .

2.1.7.2 Reference modifications [meta.trans.ref]

¹ Add the following to the table “Reference modifications”:

Table 2 — Reference modifications

Template	Comments
template<class From, class To> struct copy_reference;	The member typedef <code>type</code> names the same type as <code>add_rvalue_reference_t<To></code> if <code>is_rvalue_reference_v<From></code> , <code>add_lvalue_reference_t<To></code> if <code>is_lvalue_reference_v<From></code> , and <code>To</code> otherwise.
template<class From, class To> struct clone_reference;	The member typedef <code>type</code> names the same type as <code>copy_reference_t<From, remove_reference_t<To>></code> .

2.1.7.3 Sign modifications

[meta.trans.sign]

- ¹
- Add the following to the table “Sign modifications”:

Table 3 — Sign modifications

Template	Comments
template<class From, class To> struct copy_signedness;	The member typedef <code>type</code> names the same type as <code>make_signed_t<To></code> if <code>is_same_v<From, make_signed_t<From>></code> , <code>make_unsigned_t<To></code> if <code>is_same_v<From, make_unsigned_t<From>></code> , and <code>To</code> otherwise. <i>Requires:</i> <code>From</code> and <code>To</code> shall be (possibly cv-qualified) integral types or enumerations but not <code>bool</code> types.

2.1.7.4 Array modifications

[meta.trans.arr]

- ¹
- Add the following to the table “Array modifications”:

Table 4 — Array modifications

Template	Comments
template<class From, class To> struct copy_extent;	The member typedef <code>type</code> names the same type as <code>To[extent_v<From>]</code> if <code>rank_v<From> > 0</code> && <code>extent_v<From> > 0</code> , <code>To[]</code> if <code>rank_v<From> > 0</code> && <code>extent_v<From> == 0</code> , and <code>To</code> otherwise. <i>Requires:</i> <code>To</code> shall not be an array of unknown bound along its first dimension if <code>From</code> is an array of unknown bound along its first dimension.
template<class From, class To> struct clone_extent;	The member typedef <code>type</code> names the same type as <code>copy_extent_t<From, remove_extent_t<To>></code> . <i>Requires:</i> <code>From</code> and <code>To</code> shall not be arrays of unknown bounds along their first dimension at the same time.
template<class From, class To> struct copy_all_extents;	The member typedef <code>type</code> names the same type as <code>copy_extent_t<From, copy_all_extents_t<std::remove_extent_t<From>, To>></code> if <code>rank_v<From> > 0</code> , and <code>To</code> otherwise. <i>Requires:</i> <code>From</code> and <code>To</code> shall not be arrays of unknown bounds along their first dimension at the same time.
template<class From, class To> struct clone_all_extents;	The member typedef <code>type</code> names the same type as <code>copy_all_extents_t<From, remove_all_extents_t<To>></code> .

2.1.7.5 Pointer modifications

[meta.trans.ptr]

- ¹ Add the following to the table “Pointer modifications”:

Table 5 — Pointer modifications

Template	Comments
template<class T> struct remove_all_pointers;	The member typedef <code>type</code> names the same type as <code>remove_all_pointers_t<remove_pointer_t<T>></code> if <code>is_pointer_v<T></code> , and <code>T</code> otherwise.
template<class From, class To> struct copy_pointer;	The member typedef <code>type</code> names the same type as <code>copy_cv_t<From, add_pointer_t<To>></code> if <code>is_pointer_v<From></code> , and <code>To</code> otherwise.
template<class From, class To> struct clone_pointer;	The member typedef <code>type</code> names the same type as <code>copy_pointer_t<From, remove_pointer_t<To>></code> .
template<class From, class To> struct copy_all_pointers;	The member typedef <code>type</code> names the same type as <code>copy_pointer_t<From, copy_all_pointers_t<std::remove_pointer_t<From>, To>></code> if <code>is_pointer_v<From></code> , and <code>To</code> otherwise.
template<class From, class To> struct clone_all_pointers;	The member typedef <code>type</code> names the same type as <code>copy_all_pointers_t<From, remove_all_pointers_t<To>></code> .

2.1.7.6 Other transformations

[meta.trans.other]

- ¹ Add the following to the table “Other transformations”:

Table 6 — Other transformations

Template	Comments
template<class From, class To> struct copy_cvref;	The member typedef <code>type</code> names the same type as <code>copy_reference_t<From, copy_cv_t<remove_reference_t<From>, To>></code> .
template<class From, class To> struct clone_cvref;	The member typedef <code>type</code> names the same type as <code>copy_cvref_t<From, remove_cvref<To>></code> .

2.1.8 Logical operator traits

[meta.logical]

- ¹ No modification.

2.1.9 Endian

[meta.endian]

- ¹ No modification.